

Reviewer Pack — Free Entropy Lower Bound on Disjointness Communication Compl...

Entry 54cd565a7ba3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 06:02:56 UTC

Free Entropy Lower Bound on Disjointness Communication Complexity

Entry ID: 54cd565a7ba3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 06:02:56 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a random instance M of the DISJOINTNESS problem on n -bit inputs, the free entropy $\phi(M)$ satisfies $\phi(M) \geq c \cdot n$ for some absolute constant $c > 0$. This lower bound is tight up to polynomial factors.

Rationale (proposer's reasoning):

Free entropy captures noncommutative information structure, which may mirror the entanglement required for disjointness protocols. The conjecture links operator-algebraic invariants to communication complexity via matrix-valued distributions.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4cc79d5bbdfbd58a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = random.choice([10, 15, 20, 30, 40])
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    # Compute free entropy  $\phi(M)$ 
    def r_transform(M):
        n = len(M)
        det = 1
        for i in range(n):
            det *= (M[i][i] + 1j * sum(M[j][i] for j in range(i+1, n)))
        return math.log(abs(det))

     $\phi_M$  = r_transform(M) / (2 * math.pi)

    # Check the conjecture
    c = 0.1
    if  $\phi_M$  < c * n:
        return {
            "metric_name": "free_entropy",
            "metric_value":  $\phi_M$ ,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f" $\phi(M) / n = \{\phi_M / n\} < \{c\}$ "
        }
    else:
        return {
            "metric_name": "free_entropy",
            "metric_value":  $\phi_M$ ,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]
```

```

results = []
for seed in seeds:
    random.seed(seed)
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\" $\phi(M)$  /  $n < 0.1$ \" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8acca6bd.py", line 58, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8acca6bd.py", line 30, in run_trial
     $\phi_M$  = r_transform(M) / (2 * math.pi)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8acca6bd.py", line 28, in r_transform
    return math.log(abs(det))
                   ~~~~~^
ValueError: math domain error

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with math domain error, preventing evaluation of $\phi(M)$. The pre-registered support condition cannot be verified without successful trials. | next: Debug the `r_transform` function's determinant calculation to avoid math domain errors, particularly when `det` approaches zero

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	39467
2	preregistration	ollama_remote	qwen3:8b	0	0	12926
3	novelty	ollama_remote	qwen3:8b	0	0	9949
4	novelty	ollama_remote	qwen3:8b	0	0	7781
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11797
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6684
7	judge	ollama_remote	qwen3:8b	0	0	16706

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105311 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/54cd565a7ba3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54cd565a7ba3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/54cd565a7ba3.tar.gz` (if generated)